Technical Note — Scholarship Intelligence Crawler

1. Architecture

The system is a Python pipeline of six stages, each a separate module so it can be inspected and tested in isolation:

Discover (src/discovery.py) -> Crawl (src/fetcher.py) -> Extract (src/extract.py) -> Verify/Score (src/scoring.py) -> Store (src/store.py) -> Update (snapshot diff in src/store.py, orchestration in src/crawler.py).

A Streamlit dashboard (dashboard/app.py) reads the same SQLite database the crawler writes. One invocation of python -m src.crawler is one complete run; runs are first-class rows in the database, and every record keeps a snapshot per run.

2. Technology choices

Python 3 with requests + BeautifulSoup for crawling, pypdf for official PDF guideline documents, sqlite3 for storage and Streamlit for the dashboard. All free and open source; no paid APIs, no scraping services, and no language model anywhere in the extraction or scoring path — an intentional choice, because the assignment's core demand is that values and scores be *produced by a checkable methodology*, not generated. The fetcher is deliberately polite: per-domain delays, exponential-backoff retries and a size cap.

3. Discovery methodology

Only index pages are configured (config/seeds.json): the National Scholarship Portal's scheme listing, the Commonwealth Scholarship Commission and USIEF index pages, and foundation/CSR index pages (Inlaks, Tata Trusts, Foundation For Excellence, LIC Golden Jubilee Foundation). No scholarship record is hand-entered. From each index the crawler discovers candidate programme pages on the provider's own domain by anchor/URL evidence. The NSP listing is parsed structurally: each scheme block yields name, ministry, opening/closing dates and the URL of the official guideline PDF. Discovery therefore generalises: adding a provider means adding its index page, not writing a scraper per scholarship.

4. Extraction methodology

Extraction is rule-based. HTML pages and PDF guideline documents are reduced to text lines; field extractors then apply heading/section detection (Eligibility, Documents Required, Selection Process), keyword-anchored date search (last date / closing / open till), currency patterns for amounts, and targeted patterns for income, marks, age, category, domicile and renewal statements. Dates are normalised to ISO

format at extraction time. Every extracted value is stored with the verbatim snippet it came from (field_evidence), giving the chain database -> official source -> evidence -> value.

5. Verification methodology

A record is anchored to a primary source: the provider's own page or its official guideline document. Known aggregator domains are hard-blocked from ever being authoritative (they may only be discovery hints, and this corpus did not need them). Verification checks that the scholarship's name is actually present on the official source, that an application URL exists on an official domain, that deadline and eligibility are backed by stored evidence, and that a second source (the portal listing vs. the guideline document) does not contradict the extracted values.

6. Confidence-score methodology

Deterministic weighted rules, 100 points: official primary source 25; presence confirmed on the official source 20; deadline evidenced 15; eligibility evidenced 10; official application URL 10; information current (source fetched in the latest run) 8; no conflicts 5; field traceability (>=90% of populated fields evidenced) 7. A record is **VERIFIED only at >= 95**; below that it is **REVIEW REQUIRED**. The dashboard exposes the per-rule breakdown for every record, so any score can be audited rule by rule.

7. Anti-hallucination approach

Three mechanisms. (1) *Copy, never generate*: extractors only copy strings out of fetched source text; a field with no supporting text is stored as the literal Not specified. (2) *Evidence binding*: a value only counts toward the confidence score when its evidence snippet is stored with it. (3) *Date normalisation*: dates are compared in ISO form, so a source that merely reformats a date does not create phantom "changes" or phantom facts.

8. Change detection

Each run stores a full snapshot of every record. Consecutive snapshots are compared field by field; any difference writes a changes row with old value, new value, detection timestamp, source URL and the new evidence history is appended, never overwritten. Lifecycle status is derived from the evidenced closing date (ACTIVE / EXPIRING_SOON <= 30 days / EXPIRED). A previously known record whose official source can no longer be fetched or confirmed transitions to NO_LONGER_VERIFIABLE, and that transition is itself recorded as a change. The demonstration (docs/DEMO.md) shows two live runs of this loop, and tests/ contains fixture-based tests that pin the diff semantics (a changed deadline is detected with old/new retained; an identical re-crawl

produces no changes).